ТЗ: платёжный модуль на NestJS (тестовый проект)

Nov 7, 2026 · @ak0lda

Цель

Минимальный сервис платежей на NestJS и PostgreSQL, по которому заказчик оценит уровень кода. Объём урезан сознательно: только то, что отличает платёжный код от обычного CRUD.

Проект показывает три вещи:

- повторный запрос с тем же ключом не создаёт второй платёж;
- повторно доставленный вебхук не меняет данные второй раз;
- статус меняется в транзакции, под блокировкой строки и только по допустимым переходам.

Ориентир: 4 дня с учётом изучения NestJS. День на основы и заготовку проекта, два дня на код, день на README и подготовку к вопросам.

Стек и окружение

Стек как в вакансии, всё остальное по минимуму.

Что	Выбор	Зачем
Язык	TypeScript, режим strict	Ловит ошибки типов до запуска
Фреймворк	NestJS	Стек заказчика
База	PostgreSQL в Docker Compose, схема в init.sql	Postgres сам создаёт таблицы при первом старте, миграции не нужны
ORM	TypeORM	Блокировка строки одной строкой: setLock('pessimistic_write')
Валидация	class-validator и глобальный ValidationPipe	Проверка тела запроса
Настройки	process.env и файл .env.example	Без отдельного модуля конфигурации
Тесты	Jest из шаблона NestJS	Один unit-тест

Модель данных

Две таблицы в init.sql. Защита от дублей держится на уникальных индексах в базе, а не на проверках в коде.

Таблица payments:

Поле	Тип	Правило
id	uuid	Первичный ключ
amount	bigint	Сумма в копейках, больше 0
currency	char(3)	RUB, USD или EUR
status	varchar	pending, succeeded или failed
idempotency_key	varchar	Уникальный, обязательный
provider_payment_id	varchar	Уникальный, ID платежа у провайдера
created_at, updated_at	timestamptz	Заполняются автоматически

Таблица webhook_events:

Поле	Тип	Правило
id	uuid	Первичный ключ
event_id	varchar	Уникальный, ID события у провайдера
payment_id	uuid	Внешний ключ на payments
status	varchar	Статус из вебхука
received_at	timestamptz	Время получения

API платежей

Два эндпоинта. Провайдер не настоящий: при создании платежа сервис сам генерирует provider_payment_id вида prov_<uuid>, как будто его вернул провайдер.

POST /payments — создать платёж.

- Заголовок Idempotency-Key обязателен, без него ответ 400.
- Тело: amount (целое число копеек, больше 0) и currency (RUB, USD или EUR).

- Невалидное тело: 400 с сообщением от ValidationPipe.

Поведение по ключу:

Ситуация	Ответ
Ключ новый	201, создан платёж в статусе pending
Ключ уже был	200, тот же платёж, новый не создаётся
Два запроса с одним ключом одновременно	Один создаёт платёж, второй ловит ошибку уникального индекса (код 23505) и возвращает созданный

GET /payments/:id — получить платёж. 200 с данными или 404, если такого нет.

Вебхук провайдера

Это главная часть проекта. Вебхук обрабатывается в одной транзакции: если что-то пошло не так, не сохраняется ничего, и провайдер пришлёт его повторно.

POST /webhooks/provider

- Тело: event_id, provider_payment_id, status (succeeded или failed).
- Подпись в минимальной версии не проверяется, это пункт в README «Что бы добавил в проде».

Шаги обработки внутри транзакции:

1. Записать событие в webhook_events. Если event_id уже есть (ошибка уникального индекса), ответить 200 и ничего не менять: это повторная доставка.
2. Найти платёж по provider_payment_id с блокировкой строки (SELECT ... FOR UPDATE). Платежа нет: откатить транзакцию и ответить 404, провайдер повторит позже.
3. Проверить, допустим ли переход статуса (таблица ниже). Недопустимый: записать в лог, ответить 200, статус не трогать.
4. Обновить статус, зафиксировать транзакцию, ответить 200.

Допустимые переходы. Логика — отдельная чистая функция в common/status-transitions.ts.

Текущий статус	Пришёл статус	Результат
pending	succeeded	Обновить
pending	failed	Обновить
succeeded	failed	Игнорировать, записать в лог
failed	succeeded	Игнорировать, записать в лог
любой	тот же	Ничего не делать

Тесты

Один unit-тест на функцию переходов статусов: по проверке на каждую строку таблицы из раздела про вебхук. Запуск командой npm test.

Остальное проверяется руками по примерам curl из README:

- один и тот же Idempotency-Key дважды даёт один платёж;
- вебхук succeeded меняет статус;
- тот же event_id повторно ничего не меняет;
- failed после succeeded не меняет статус.

Структура репозитория и README

Два модуля и общая папка. Проверяющий должен за минуту понять, где что лежит.

```
src/
  main.ts                  глобальный ValidationPipe
  app.module.ts            подключение TypeORM из process.env
  payments/
    payments.module.ts
    payments.controller.ts
    payments.service.ts
    dto/create-payment.dto.ts
    entities/payment.entity.ts
  webhooks/
    webhooks.module.ts
    webhooks.controller.ts
    webhooks.service.ts
    entities/webhook-event.entity.ts
  common/
```

```
    status-transitions.ts
    status-transitions.spec.ts
init.sql
docker-compose.yml         монтирует init.sql в /docker-entrypoint-initdb.d
.env.example
README.md
```

init.sql выполняется только при первом старте с пустой базой. Если поменяли схему, базу нужно пересоздать: docker compose down -v.

В README:

- запуск с нуля: docker compose up -d, npm install, npm run start:dev;
- примеры curl: создать платёж дважды с одним ключом, отправить вебхук дважды;
- «Почему так», по строке на пункт: копейки вместо float; уникальный индекс вместо проверки в коде; зачем FOR UPDATE; почему при ошибке вебхук получает не 2xx;
- «Что бы добавил в проде»: проверка подписи вебхука HMAC по сырому телу через timingSafeEqual; ответ 422 на тот же ключ с другим телом; миграции вместо init.sql; e2e-тесты.

Чего не делать и когда готово

Всё, что не помогает показать идемпотентность и транзакцию, в минимальную версию не идёт.

Не делать: авторизацию, настоящего провайдера, фронтенд, очереди, подпись вебхука, миграции, e2e-тесты, абстракции «на будущее».

Готово, когда:

- [ ] сервис поднимается с нуля по README
- [ ] npm test проходит
- [ ] в репозитории нет секретов, есть .env.example
- [ ] Prettier и ESLint без ошибок
- [ ] несколько осмысленных коммитов, а не один init
- [ ] вы отвечаете на вопросы ниже без подсказок

Вопросы для самопроверки

Эти вопросы почти наверняка зададут по коду. Ответьте на каждый своими словами до отправки; если не получается, вернитесь к соответствующему месту в коде.

1. Два запроса с одним ключом пришли одновременно. Почему не будет двух платежей?
2. Тот же ключ, но другая сумма. Что вернёте и почему не просто старый платёж?
3. Вебхук пришёл раньше, чем платёж сохранился. Какой ответ и что сделает провайдер?
4. Почему деньги в копейках целым числом, а не float?
5. Зачем SELECT ... FOR UPDATE, если всё и так в транзакции?
6. Почему подпись проверяется по сырому телу, а не по распарсенному JSON?
7. Почему timingSafeEqual, а не обычное сравнение строк?
8. Сервис упал между записью события и обновлением статуса. Что будет с данными?
9. Как добавить второго провайдера, не переписывая модуль вебхуков?